

SYSTEM-ON-CHIP DESIGN

Connecting a RISC-V Processor to Multiple Peripherals over an APB Bus

Design & Verification Document

RTL Architecture · RISC-V / APB Bridge · UART Peripheral · Functional Verification · Vivado Synthesis

Prepared from the project repository

SoC-Design-Connecting-RISC-V-Processor-with-Multiple-peripherals-using-APB-Bus

Table of Contents

- 1. Introduction & Overview 3
- 2. System Architecture 5
- 3. RISC-V Processor Core 6
- 4. RISC-V ↔ APB Bridge (RISC_APB_Wrapper) 10
- 5. APB Master 11
- 6. APB Decoder & Address Map 12
- 7. UART Peripheral 12
- 8. Verification & Test Results 14
- 9. Synthesis & Implementation (Vivado) 21
- 10. Repository Structure 22
- 11. Future Work 23

1. Introduction & Overview

A System on Chip (SoC) integrates a processor, memory, and input/output peripherals onto a single design, connected through a shared on-chip communication bus. This project implements such a system: a single-cycle RISC-V processor core is connected to a UART peripheral through an AMBA APB (Advanced Peripheral Bus) interconnect, allowing the processor to communicate with the outside world using standard lw / sw memory instructions.

Rather than adding a dedicated memory-mapped peripheral interface to the processor's datapath, the design intercepts specific load/store instructions at the boundary between the core and its data memory. A bridging module — the RISC-APB Wrapper — inspects every lw/sw instruction; if its effective address falls above a defined threshold, the instruction is diverted onto the APB bus instead of completing against local data memory. This keeps the RISC-V core largely unmodified while giving it access to memory-mapped peripherals.

1.1 Design Goals

- Implement a working single-cycle RISC-V processor core (a subset of RV32I).
- Bridge the processor to a standard APB master/slave interconnect without redesigning its control path.
- Decode APB transactions to the correct peripheral using an address-range decoder.
- Implement a fully functional UART peripheral (transmit + receive, configurable baud rate, parity and framing/overflow error detection) as the first peripheral on the bus.
- Verify the integrated system end-to-end in simulation, and carry the design through Vivado elaboration, synthesis and implementation.

1.2 Document Scope

This document focuses primarily on the RTL architecture and design decisions of each block in the system, and secondarily presents the functional verification evidence (simulation waveforms) and physical implementation views (Vivado) captured from the project repository. Each major module is described with its role, key internal logic, and interface signals.

1.3 High-Level System View

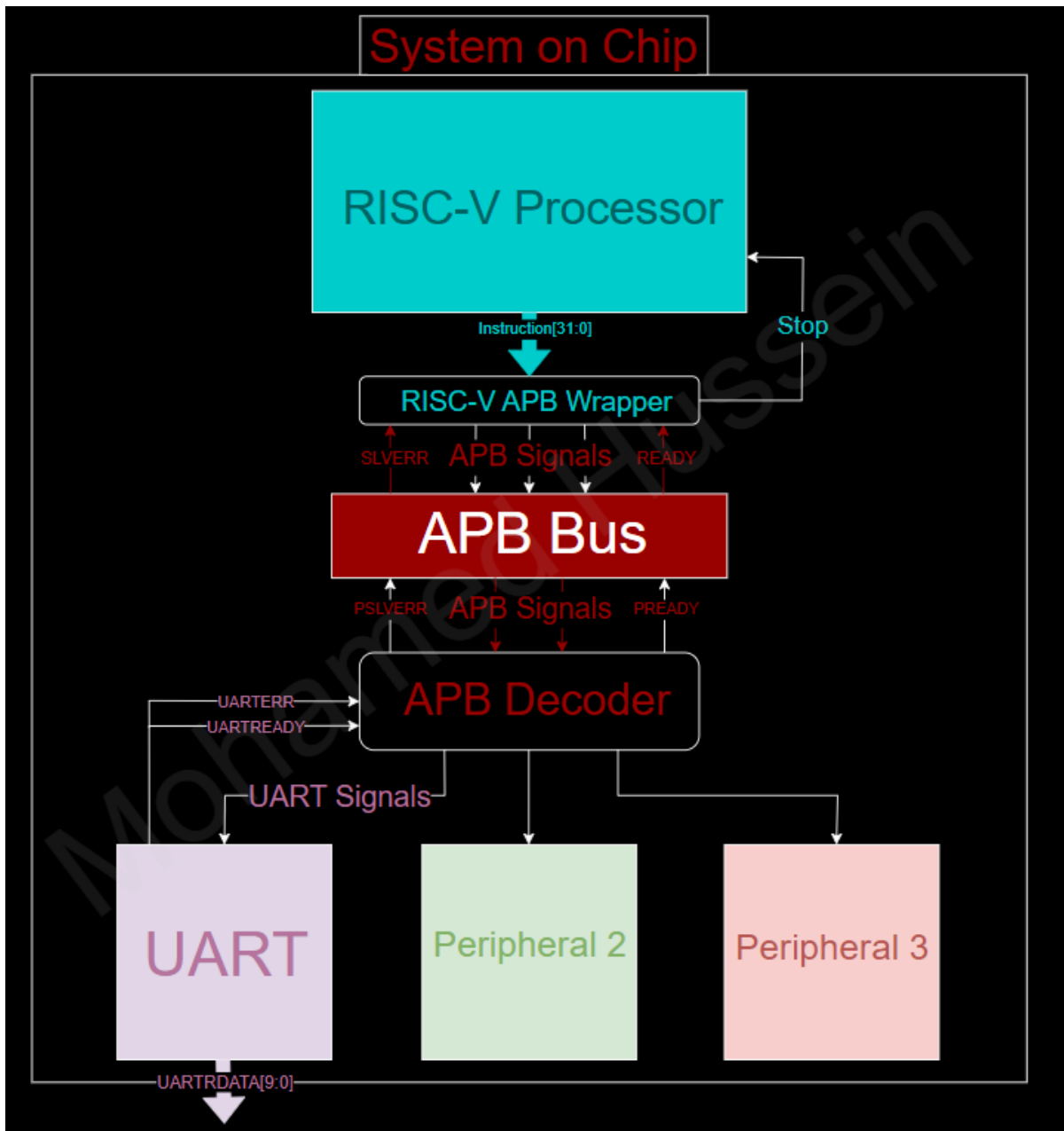


Figure 1.1 — Top-level SoC block diagram: RISC-V core, RISC-APB wrapper, APB bus, APB decoder and peripherals.

As shown above, the RISC-V processor issues instructions to the RISC-APB Wrapper. When a load/store instruction targets the APB address space, the wrapper freezes the processor's program counter (stop) and drives an APB transaction through the APB Master. The APB Decoder routes the transaction to the correct peripheral based on address range and translates the generic APB handshake into peripheral-specific control signals. In the current release, one peripheral — UART — is implemented, with the address decoder already structured to support two additional peripherals.

2. System Architecture

The SoC top level (SoC.v) instantiates and wires together five structural blocks: the RISC-V processor wrapper, the RISC-APB bridging wrapper, the APB master, the APB decoder, and the UART peripheral. All blocks share a single clk / rst pair; the design is fully synchronous with a single clock domain for the processor and APB bus, and a derived baud-rate clock domain internal to the UART.

2.1 Vivado Elaboration Schematic

The figure below is the actual elaborated RTL schematic generated by Vivado, showing the five structural instances of the SoC top module and every port connecting them — this is the ground-truth structural view of the design as it was synthesized.

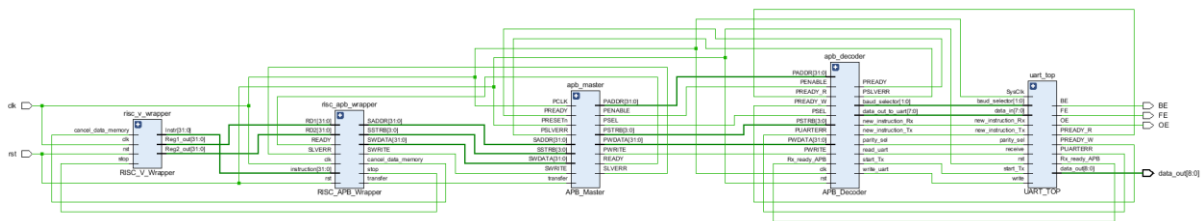


Figure 2.1 — Vivado elaboration schematic of the top-level SoC (*risc_v_wrapper* → *risc_apb_wrapper* → *apb_master* → *apb_decoder* → *uart_top*).

2.2 Design Hierarchy

- SoC (SoC TOP/SoC.v) — top-level integration
 - RISC_V_Wrapper — single-cycle RISC-V core
 - PC_clk, Adder4, Adder_PC, Inst_Memory, Control_Unit, Reg_File_Model, Extention_Unit, Mux2_1 x2, Mux3_1, ALU, Data_Mem
 - RISC_APB_Wrapper — instruction-trapping bridge / FSM
 - APB_Master — AMBA APB master FSM
 - APB_Decoder — address decode + peripheral signal translation
 - UART_TOP — UART peripheral
 - Baud_Rate_Generator
 - Tx_Unit → Tx_Transmitter, Tx_FIFO, Parity_Selector
 - Rx_Unit → Rx_FIFO, Rx_Receiver, Error_Detector

2.3 Top-Level Ports

Port	Dir.	Width	Description
clk	in	1	System clock — drives the RISC-V core, APB bus and (via the baud generator) the UART.
rst	in	1	Asynchronous system reset, shared by all sub-blocks.

Port	Dir.	Width	Description
data_out	out	9	Parallel data received by the UART (8 data bits + 1 parity bit).
OE / BE / FE	out	1 each	UART Overrun / Break / Framing error flags.

2.4 Internal Signal Groups

The full internal signal list is documented in the source (SoC.v); the table below groups the signals by the interface they belong to rather than listing all of them individually.

Interface	Representative signals	Purpose
Core ↔ Bridge	Instr, Reg1_out, Reg2_out, stop, cancel_data_memory	Exposes the fetched instruction and register operands to the bridge; lets the bridge freeze the PC and suppress the local data-memory access.
Bridge ↔ APB Master	transfer, SWRITE, SADDR, SWDATA, SSTRB, READY, SLVERR	The bridge's simplified view of a bus transaction, driven into the APB master.
APB Master ↔ Decoder	PSEL, PENABLE, PWRITE, PADDR, PWDATA, PSTRB, PREADY, PSLVERR	Standard AMBA APB signal set.
Decoder ↔ UART	write_uart, read_uart, parity_sel, baud_selector, data_out_to_uart, start_Tx, Rx_ready_APB, PREADY_W/R, PUARTERR	Peripheral-specific control/status translated from the generic APB signals.

3. RISC-V Processor Core

The processor is a single-cycle RV32I-style core, structured after the classic Harris & Harris single-cycle datapath. Every instruction completes in one clock cycle: fetch, register read, ALU execute, memory access and register write-back all occur combinationally within the same cycle, with only the PC and register file being clocked storage elements.

3.1 Datapath

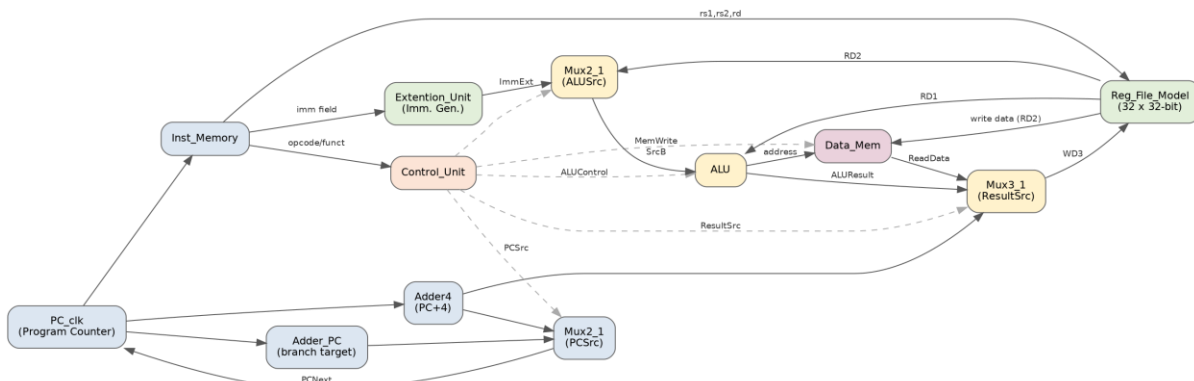


Figure 3.1 — Single-cycle RISC-V datapath (solid arrows: data; dashed arrows: control signals from Control_Unit).

3.2 Instruction Set Architecture (Instruction Formats)

RISC-V is a load/store, fixed-width instruction set: every instruction is exactly 32 bits wide, and that fixed width is sliced into a small number of standard field layouts depending on what the instruction needs to encode. The core implements the five formats below — R-type (register-register ALU ops), I-type (immediate ALU ops and loads), S-type (stores), B-type (branches) and J-type (unconditional jumps, added as this project's enhancement over the baseline single-cycle design). Every field's position and width is fixed within a format, which is precisely what lets Inst_Memory hand a raw 32-bit word to Control_Unit and have opcode / funct3 / funct7 fall out at constant bit positions, with no variable-length parsing of any kind.

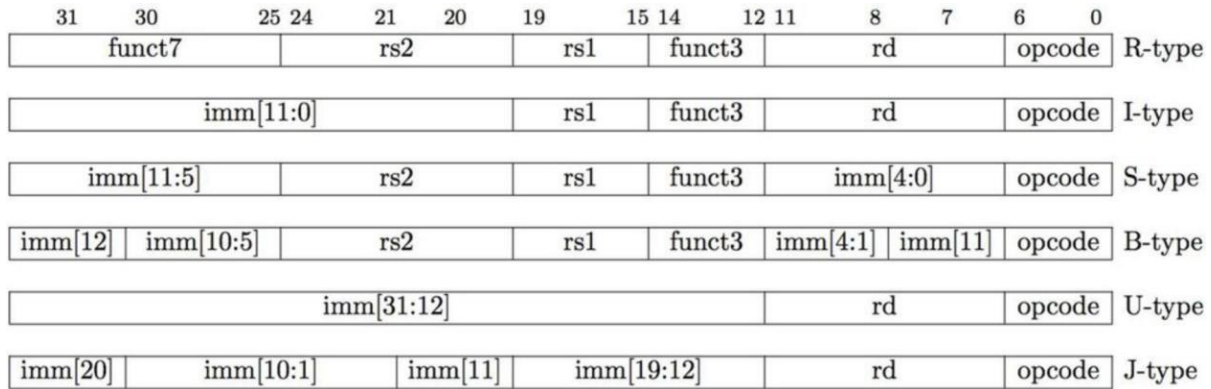


Figure 3.2 — RISC-V instruction formats: R, I, S, B, U and J type field layouts (bit widths shown beneath each field).

The Extention_Unit reproduces exactly these layouts in reverse for the immediate fields: given ImmSrc (selected from the opcode), it re-assembles the scattered imm bits of an S-, B- or J-type word into a single sign-extended 32-bit value — this is why, for example, the branch and jump immediates are encoded across several non-contiguous bit ranges instead of one contiguous field: the encoding keeps rs1/rs2/rd and opcode at constant bit positions across every format, at the cost of a slightly scrambled immediate.

3.3 Module Breakdown

Module	Role
PC_clk	Clocked program-counter register; loads PCNext on every rising edge.
Inst_Memory	Combinational instruction ROM, indexed by PC (backed by Inst_Mem.dat).
Adder4 / Adder_PC	Compute PC+4 (sequential next instruction) and PC+immediate (branch/jump target).
Control_Unit	Main + ALU decoders; derives all control signals from opcode / funct3 / funct7.
Reg_File_Model	32 × 32-bit register file, two combinational read ports (RD1, RD2), one clocked write port (WD3/WE3).
Extention_Unit	Sign-extends the immediate field according to instruction type (I/S/B/J).
ALU	Add, subtract, AND, OR, set-less-than; also produces the Zero flag used for branches.
Data_Mem	Data memory, written on MemWrite, read combinationally (backed by D_Mem.dat).

Module	Role
Mux2_1 / Mux3_1	Operand and result-path multiplexers (ALUSrc, ResultSrc, PCSrc).
RISC_V_Wrapper	Structural top module instantiating and connecting all of the above.

3.4 Supported Instructions

The main decoder in Control_Unit currently recognizes the following RV32I opcodes:

Opcode	Instruction class	Example	Notes
0000011	lw (load word)	lw x1, imm(x9)	Diverted to APB when effective address ≥ 1000 .
0100011	sw (store word)	sw x4, imm(x9)	Diverted to APB when effective address ≥ 1000 .
0110011	R-type	add, sub, and, or, slt	ALUOp = 2'b10, funct3/funct7 select the ALU op.
0010011	I-type (ALU imm.)	addi, andi, ori, slti	Same ALU decode path as R-type, ALUSrc = 1.
1100011	beq (branch)	beq x1, x2, label	PCSrc = Branch & Zero.
1101111	jal (jump & link)	jal x1, label	Unconditional; PCSrc = Jump.

Note on ALU control: within the R-/I-type ALUOp class, funct3 = 000 selects add or sub (distinguished by {OP[5],funct7}), funct3 = 010 selects slt, funct3 = 110 selects or, and funct3 = 111 selects and.

3.5 SoC-Specific Modification: cancel_data_memory

To let the same lw/sw instructions optionally target the APB bus instead of local data memory, the Control_Unit accepts an extra cancel_data_memory input from the bridge. When asserted for a store, MemWrite is forced low so the local Data_Mem is not corrupted; when asserted for a load, ResultSrc is forced to a don't-care so the register write-back does not sample stale data-memory output — the loaded value instead arrives later via the APB path. This is the only change made to the otherwise standard single-cycle core.

3.6 From Assembly to Machine Code: Programming the Core

A single-cycle core like this one never “interprets” assembly at run time — it has no software, no fetch-decode-execute loop implemented in code, nothing that parses text. Assembly mnemonics such as lw, sw or beq exist purely to help a human write and read a program; the assembler's real job is to translate each line into one fixed 32-bit binary word ahead of time. It is that word, and only that word, that the processor ever sees.

Concretely, in this design, the 32-bit words the processor executes are pre-loaded straight into Inst_Memory (backed by Inst_Mem.dat) before simulation starts. When the core reads word N from instruction memory, it is reading the literal machine-code encoding of instruction N — there is no translation step happening in hardware. The opcode / funct3 / funct7 fields Control_Unit decodes (Section 3.2) are just fixed bit ranges of that pre-

assembled word; “running the assembly” and “feeding fixed binary words through the datapath one per cycle” are the same thing.

The companion RISC-V core project (RISC-V Single Cycle Implementation) that this SoC's processor is based on verifies exactly this point with a short hand-assembled test program, reproduced below, that exercises one instruction of each type (I, S, R, B, J):

```

1 //assembly code to test functionality
2
3 //1: I-type
4 //2: S-type
5 //3: R-type
6 //4: B-type
7 //5: J-type
8 //6: I-type
9
10 lw x6, -4(x9) //Address 0x0000 (will assign the value 9 from data memory in x6)
11 sw x6, 8(x9) //Address 0x0004 (will print the value 9 at address 3 in the Data Memory)
12 or x4, x5, x6 //Address 0x0008
13 beq x4, x4, 32 //Address 0x000C (this will increment the pc by 16 which increments the address of instruction memory by 4)
14 jal x10, 128 //Address 0x001C (this will increment the PC by 64 which will increment the instruction memory address by 16)
15 addi x1, x0, 8 //Address 0x005C

```

Figure 3.3 — Baseline test program: one instruction of each type (I/S/R/B/J), hand-assembled to fixed machine-code words in *Inst_Mem.dat*.

With $x_9 = 4$, $x_5 = 6$ and data-memory word 0 pre-loaded with the value 9, this program is expected to produce the following results — each of which was confirmed against the simulated waveforms in the referenced processor's own verification section:

Instruction	Type	Expected result
lw x6, -4(x9)	I	x6 loaded with 9, the value already sitting in data memory.
sw x6, 8(x9)	S	Value 9 stored to data-memory word 3 ((4+8)/4).
or x4, x5, x6	R	$x_4 = x_5 \mid x_6 = 0110 \mid 1001 = 0xF$ (15).
beq x4, x4, 32	B	Branch taken (equal); PC jumps ahead by 16 words (4 x 32-bit words).
jal x10, 128	J	PC jumps ahead by 64 words; x10 latched with the return address, PC+4.
addi x1, x0, 8	I	$x_1 = 0 + 8 = 8$, confirming execution resumed correctly after the jump.

This SoC design reuses that same baseline program as the first instructions in its own instruction memory, precisely to re-confirm that the underlying RV32I datapath is still functionally correct before layering the APB/UART-specific behaviour on top of it — see Section 8.2, where the same six instructions are shown executing correctly ahead of the SoC's own trapped lw/sw scenarios.

Beyond hand-written assembly: on a project of any real size, nobody hand-assembles instructions like this. Instead, a program is written in a high-level language such as C and compiled by a RISC-V cross-compiler (e.g. the riscv32-unknown-elf-gcc toolchain) straight down to the same RV32I machine code shown above — the compiler is simply a much more sophisticated version of the same translation step. Because the processor only ever consumes fixed 32-bit words from instruction memory, it makes no distinction between a word that was typed by hand and one that a compiler generated from C source; the RTL described in this document would run either without any modification.

4. RISC-V ↔ APB Bridge (RISC_APB_Wrapper)

This module is the key piece of SoC-specific glue logic: it inspects every instruction the core fetches and decides, purely from the opcode and the computed effective address, whether the instruction should be completed locally (normal single-cycle behaviour) or redirected onto the APB bus.

4.1 Address-Trapping Condition

- For sw (opcode 0100011): if {instruction[31:25], instruction[11:7]} + RD1 $\geq$ 1000, and the top nibble of the instruction is not 4'b1111 (used as an escape/guard pattern), the store is routed to APB.
- For lw (opcode 0000011): if instruction[31:20] + RD1 $\geq$ 1000 under the same guard condition, the load is routed to APB.
- All other instructions, and lw/sw with a low effective address, execute exactly as in the unmodified single-cycle core.

4.2 Internal FSM

The bridge is a 4-state Moore machine. IDLE is the default state on every cycle the processor is running normal instructions. On detecting a trapped lw or sw, it moves to WAIT_R or WAIT_W respectively, asserting transfer, SWRITE, SADDR and (for stores) SWDATA for the APB master to consume. It remains there until READY is returned by the APB master, then transitions through READY_WAIT back to IDLE once READY drops again — this extra state exists to safely absorb the trailing cycle of the APB handshake before releasing the core.

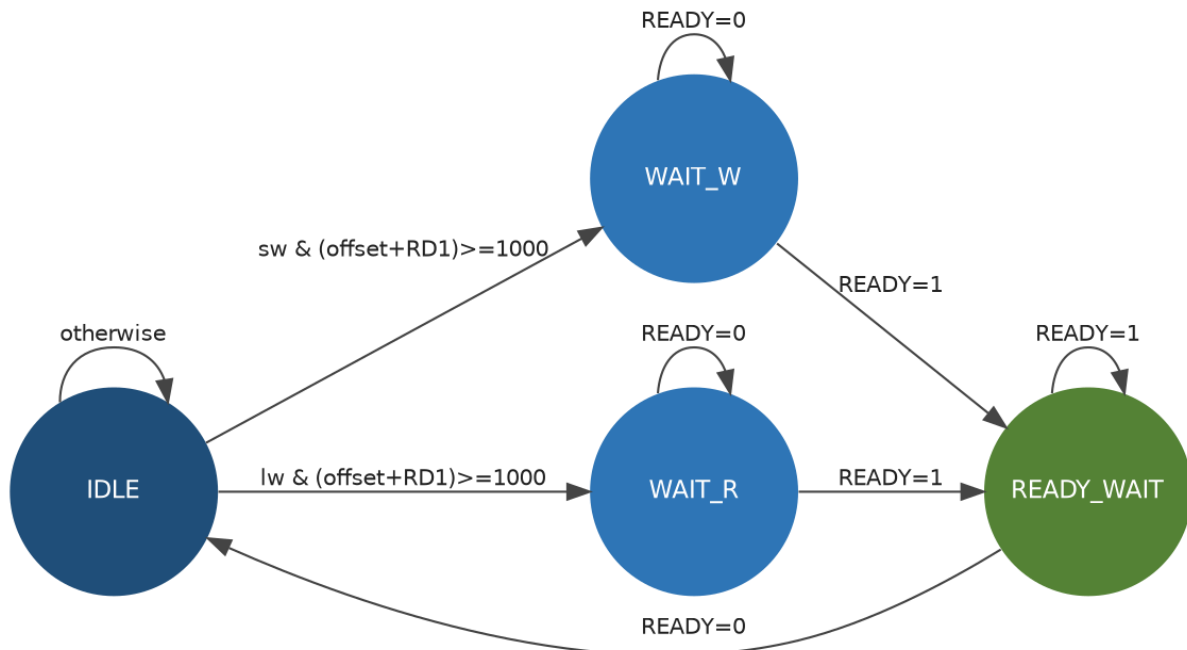


Figure 4.1 — RISC_APB_Wrapper state machine.

4.3 Core Freeze / Resume

The stop output is combinationaly derived from the next-state value ($\text{stop} = (\text{ns} \neq \text{IDLE})$): whenever the FSM is about to be, or currently is, outside IDLE, stop is asserted, which freezes the PC (Adder4 / Adder_PC hold their outputs) so the RISC-V core pauses on the trapping instruction until the APB transaction completes. cancel_data_memory follows a similar but latched (flag-based) condition so that the local Data_Mem access tied to the trapping instruction is suppressed for its entire dwell in the FSM.

4.4 Interface Summary

Signal	Dir.	Description
instruction[31:0]	in	Current instruction fetched by the core.
RD1 / RD2	in	rs1 / rs2 register values from the core's register file.
READY / SLVERR	in	Completion / error status returned from the APB master.
stop	out	Freezes the RISC-V PC while an APB transaction is in flight.
transfer, SWRITE, SADDR, SWDATA, SSTRB	out	Simplified transaction request driven into the APB master.
cancel_data_memory	out	Suppresses the core's local data-memory write/read for the trapped instruction.

5. APB Master

APB_Master implements the standard 3-state AMBA APB master state machine (IDLE → SETUP → ACCESS), converting the bridge's simple request/ready handshake into fully-formed APB signalling (PSEL, PENABLE, PADDR, PWDATA, PSTRB, PWRITE) for the decoder / peripheral side, per the APB protocol's requirement that PENABLE assert exactly one cycle after PSEL in every transfer.

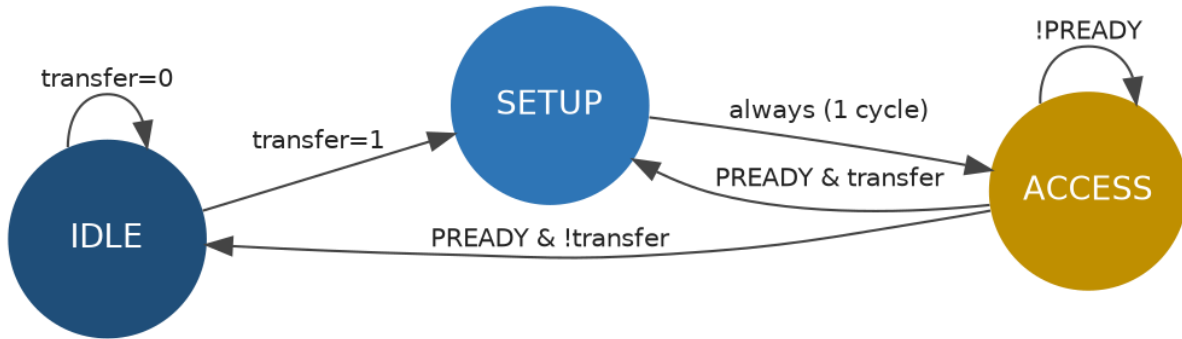


Figure 5.1 — APB_Master state machine (SETUP is always exactly one clock cycle, as required by the APB protocol).

- IDLE — PSEL and PENABLE are held low; the bus is idle until transfer is asserted by the bridge.
- SETUP — PSEL asserts and PADDR / PWDATA / PWRITE / PSTRB are driven from the bridge's SADDR / SWDATA / SWRITE / SSTRB; PENABLE stays low for exactly this one cycle.
- ACCESS — PENABLE asserts alongside PSEL. The state holds until the selected peripheral raises PREADY; on PREADY the FSM either returns to IDLE (no further transfer pending) or moves straight back to SETUP for a back-to-back transaction.

The module exposes READY and SLVERR to the bridge as direct combinational passes of PREADY and PSLVERR — the master itself introduces no additional wait states beyond what the selected peripheral requires.

6. APB Decoder & Address Map

APB_Decoder sits between the generic APB master and the peripheral-specific interfaces. It performs two jobs: (1) selecting which peripheral a transaction is addressed to, based on PADDR, and (2) translating the generic APB read/write handshake into the specific control signals each peripheral expects (and translating the peripheral's status back into PREADY / PSLVERR).

6.1 Address Map

Address range	Peripheral	Status in this release
$1000 \leq \text{PADDR} < 2000$	Peripheral 1 — UART	Implemented (write_uart / read_uart / baud_selector / parity_sel / data_out_to_uart)
$2000 \leq \text{PADDR} < 3000$	Peripheral 2	Reserved for a future peripheral
$3000 \leq \text{PADDR} < 4000$	Peripheral 3	Reserved for a future peripheral

6.2 UART Transaction Translation

- Write (PWRITE = 1): write_uart is asserted and PWDATA[7:0] is latched into data_out_to_uart for the Tx path.
- Read (PWRITE = 0): the decoder first drives start_Tx low to begin serializing the byte already sitting in the Tx FIFO across to the Rx FIFO (this design loops transmit and receive internally — see Section 7). Once Rx_ready_APB indicates the byte has arrived in the Rx FIFO, read_uart is asserted for one cycle (guarded by an internal lock so it pulses only once per read) and start_Tx returns high.
- PREADY is the logical OR of the UART's write-ready and read-ready flags (PREADY_W, PREADY_R); PSLVERR is a direct pass of the UART's combined error flag (PUARTERR).
- new_instruction_Tx / new_instruction_Rx (both tied to PREADY_R) tell the Tx/Rx FIFOs' internal serial bit counters to reset in preparation for the next transaction.

7. UART Peripheral

UART_TOP is the first (and currently only implemented) peripheral on the APB bus. Rather than requiring an external physical UART loopback for verification, the Tx and Rx datapaths are wired directly to each other on-chip (piso_out from the Tx FIFO feeds the Rx FIFO's serial input) — a store to the UART's address range serializes data out of the Tx FIFO, and a subsequent load pulls the same data back in through the Rx FIFO, exercising the full serial datapath without external hardware.

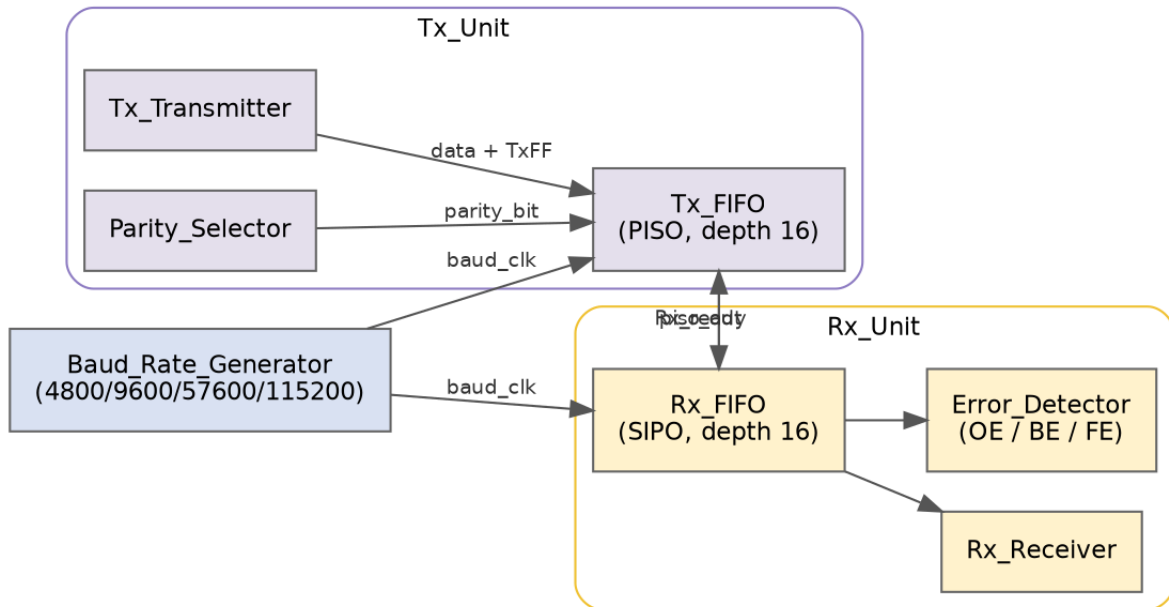


Figure 7.1 — UART internal block diagram.

7.1 Baud Rate Generator

Baud_Rate_Generator derives a toggling baud_clk from the 50 MHz system clock using a free-running divider, with 16× oversampling (the SAMPLE parameter) for accurate mid-bit sampling on the receive side. baud_selector chooses among four standard rates:

baud_selector	Rate
00	4800 baud
01 (default)	9600 baud
10	57600 baud
11	115200 baud

7.2 Transmit Path (Tx_Unit)

- Tx_Transmitter — accepts an 8-bit parallel byte on write and hands it to the Tx FIFO.
- Parity_Selector — computes the parity bit for the byte according to parity_sel.
- Tx_FIFO — a 16-deep, 9-bit-wide (8 data + 1 parity) FIFO that also performs parallel-in/serial-out shifting, driving the serial line (piso_out) at the baud rate; the TxFF flag signals a full FIFO.

7.3 Receive Path (Rx_Unit)

- Rx_FIFO — 16-deep, 12-bit-wide (8 data + 1 parity + 3 error bits) FIFO performing serial-in/parallel-out reconstruction of incoming bits at the baud rate.
- Rx_Receiver — reads completed bytes out of the Rx FIFO towards data_out (9 bits: data + parity) once available.
- Error_Detector — derives Overrun (OE), Break (BE) and Framing (FE) error flags from the reconstructed frame, combined into PUARTERR for the APB decoder.

7.4 Handshake Signals Back to the SoC

Signal	Meaning
PREADY_W	Tx FIFO has accepted/stored the written byte — a write transaction can complete.
PREADY_R	Data has fully arrived in the Rx FIFO and is available at data_out — a read transaction can complete.
Rx_ready_APB	Rx FIFO has signalled it is ready to receive from the Tx FIFO, used by the decoder to sequence start_Tx.

8. Verification & Test Results

The integrated SoC was verified in simulation using SoC_tb.sv (with SoC_DO.do driving the run in ModelSim/Questa). The testbench applies a sequence of RISC-V instructions and observes the interaction across the core, the RISC-APB bridge, the APB decoder, and both UART FIFOs.

8.1 Waveform Legend

Across the captured waveforms, selected signals are grouped and colour-coded by the block they belong to, to make the cross-module handshake easier to follow:

Colour	Signal group
Black	SoC / testbench top-level signals
Green	RISC_APB_Wrapper signals
Purple	Tx_FIFO signals
Gold	Rx_FIFO signals
Blue	Data memory (inside RISC_V_Wrapper)

8.2 Baseline Core Execution

The first six instructions loaded into Inst_Memory are the same one-instruction-per-type baseline program introduced in Section 3.6 (lw, sw, or, beq, jal, addi) — reused here to re-confirm the unmodified RISC-V datapath is functionally correct with no APB activity, before any peripheral traffic begins. From the seventh instruction onward, the SoC-specific behaviour (address trapping, APB transactions, UART traffic) is exercised. As Figure 8.1 below shows, the instruction stream is once again just a sequence of fixed 32-bit words read out of instruction memory — assembled ahead of time, not interpreted on the fly.

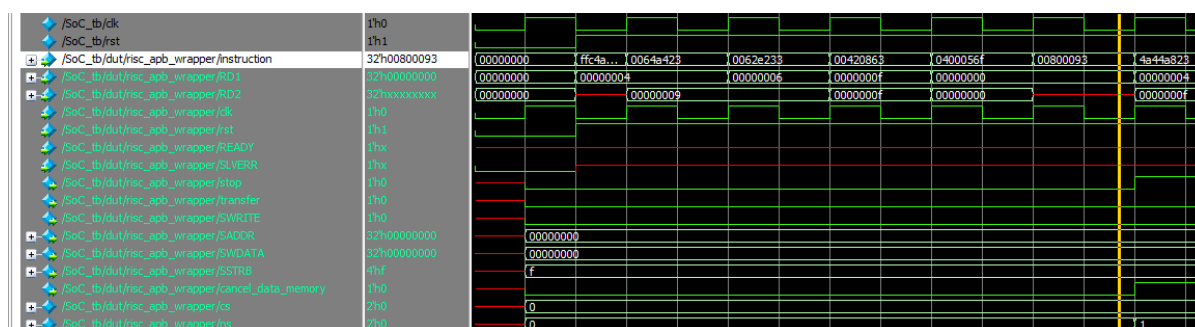


Figure 8.1 — Instruction trace: the core executes the six baseline instructions from Section 3.6 before the first trapped lw/sw.

8.3 Scenario 1 — Store to UART (sw x4, 1200(x9))

This instruction stores the value held in x4 (0xF) to address 1200, inside the UART's address range. The bridge freezes the core (stop asserted) and routes the store through the APB master to the decoder, which asserts write_uart and loads the byte into the Tx FIFO.





Figure 8.4 — Data memory at word address 301 (1204/4) remains empty, confirming the store was correctly diverted away from local memory.

8.4 Scenario 2 — Back-to-Back Store (sw x6, 1010(x9))

Immediately following Scenario 1, a second store (value 0x9 from x6) is issued to a different UART-range address. This demonstrates that consecutive trapped instructions are handled smoothly, one after another, without extra idle cycles between them.

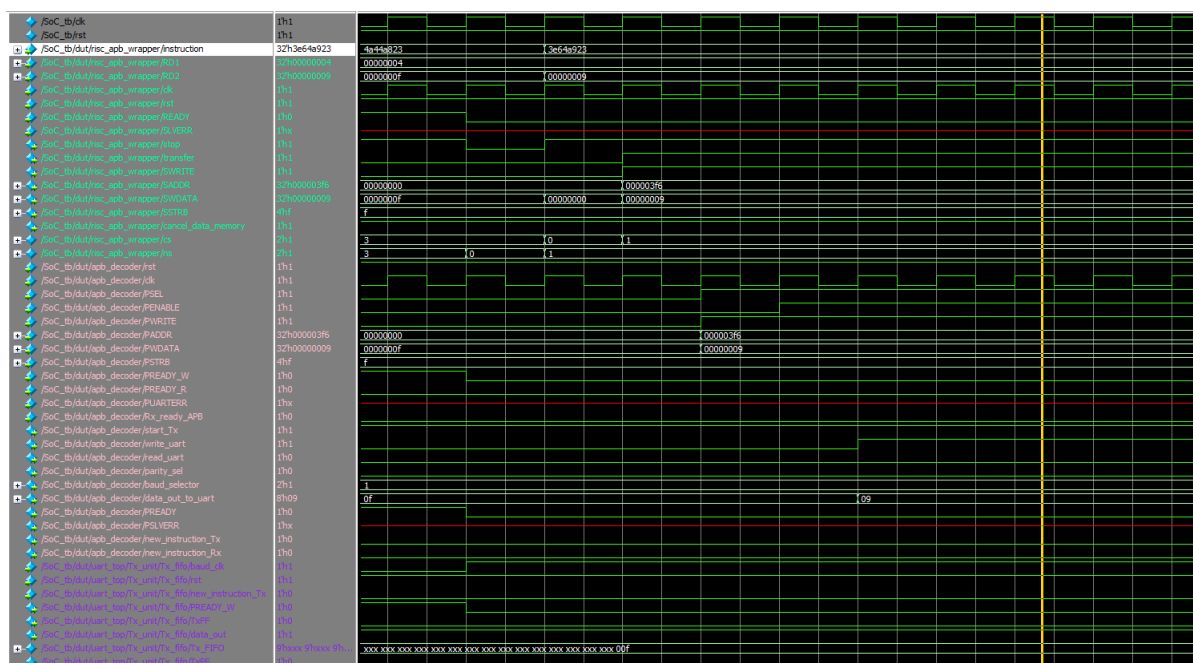
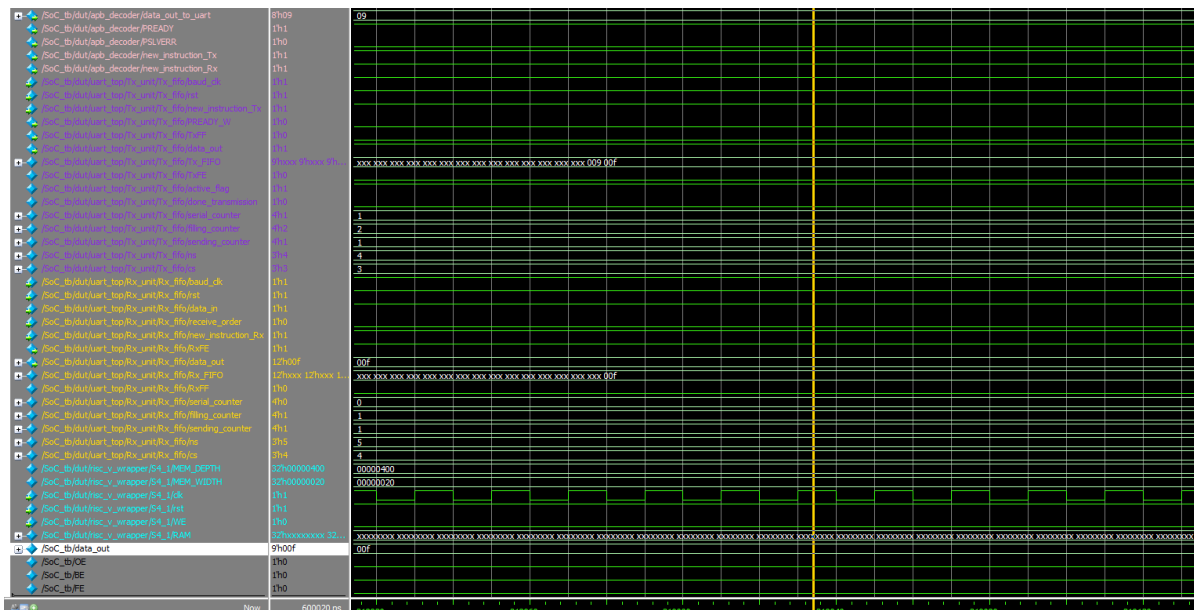
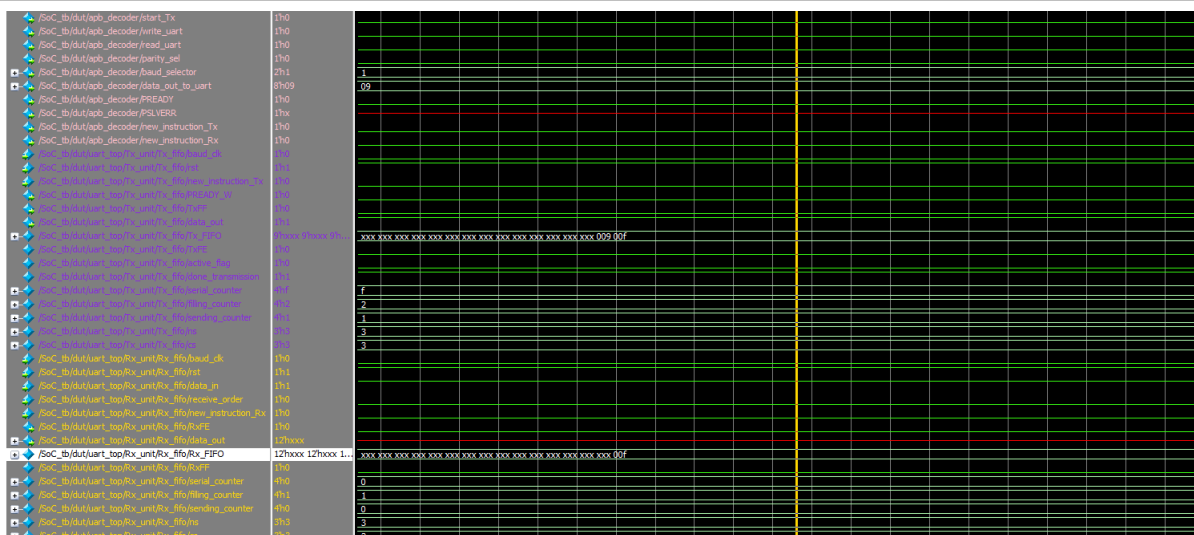


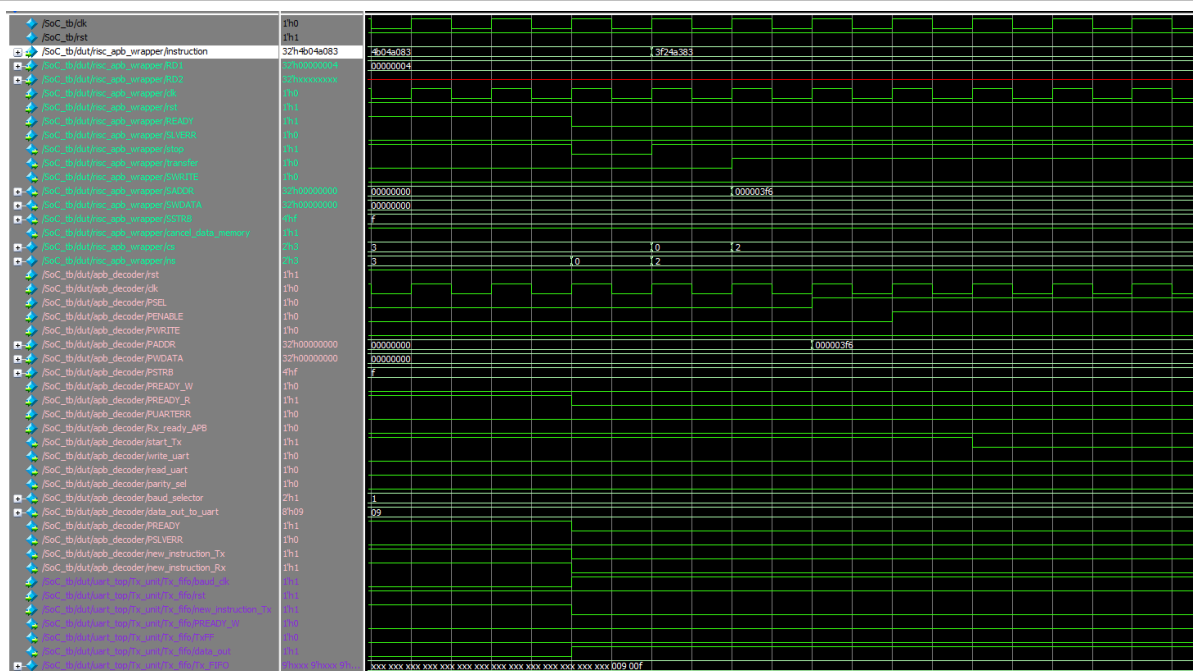
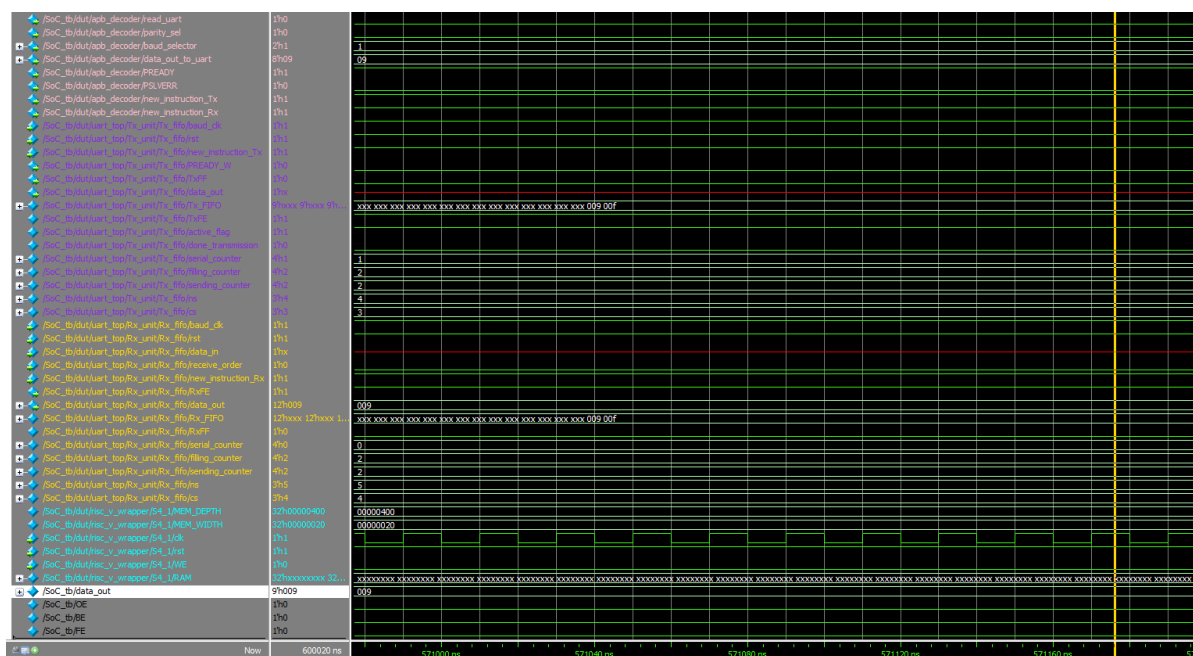
Figure 8.5 — The second store (sw x6, 1010(x9)) begins executing immediately after the first completes.



A load from the UART's address range first triggers internal serialization from the Tx FIFO across to the Rx FIFO (looped on-chip), then the reconstructed byte becomes available at data_out.





Figure 8.10 — `lw x7, 1010(x9)` is issued.Figure 8.11 — The Rx FIFO holds the new value, reflected on `data_out`.

8.7 Full-Run Waveform

The two captures below show the complete simulated sequence across all four scenarios in a single continuous run, illustrating the repeated IDLE → WAIT → READY_WAIT cycle of the bridge FSM as each trapped instruction is processed.



9. Synthesis & Implementation (Vivado)

Beyond functional simulation, the design was carried through Vivado's elaboration, synthesis and implementation flow to confirm it is structurally sound and physically realizable on the target device.

9.1 Synthesized Netlist

The synthesis engine maps the RTL to technology primitives and preserves the module hierarchy visible in Figure 2.1, now expressed as an interconnected gate/register-level netlist.

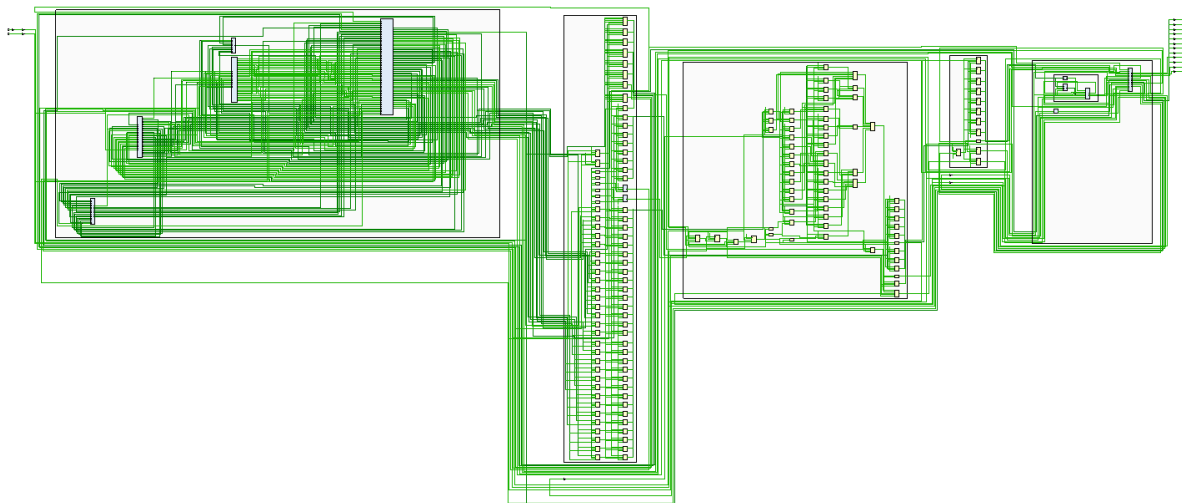


Figure 9.1 — Post-synthesis schematic view.

9.2 Implementation / Device View

The design was placed and routed onto the target device. The device floorplan below shows the resulting placement, confirming the design fits comfortably within the available fabric with room to spare for the additional peripherals planned in future releases.

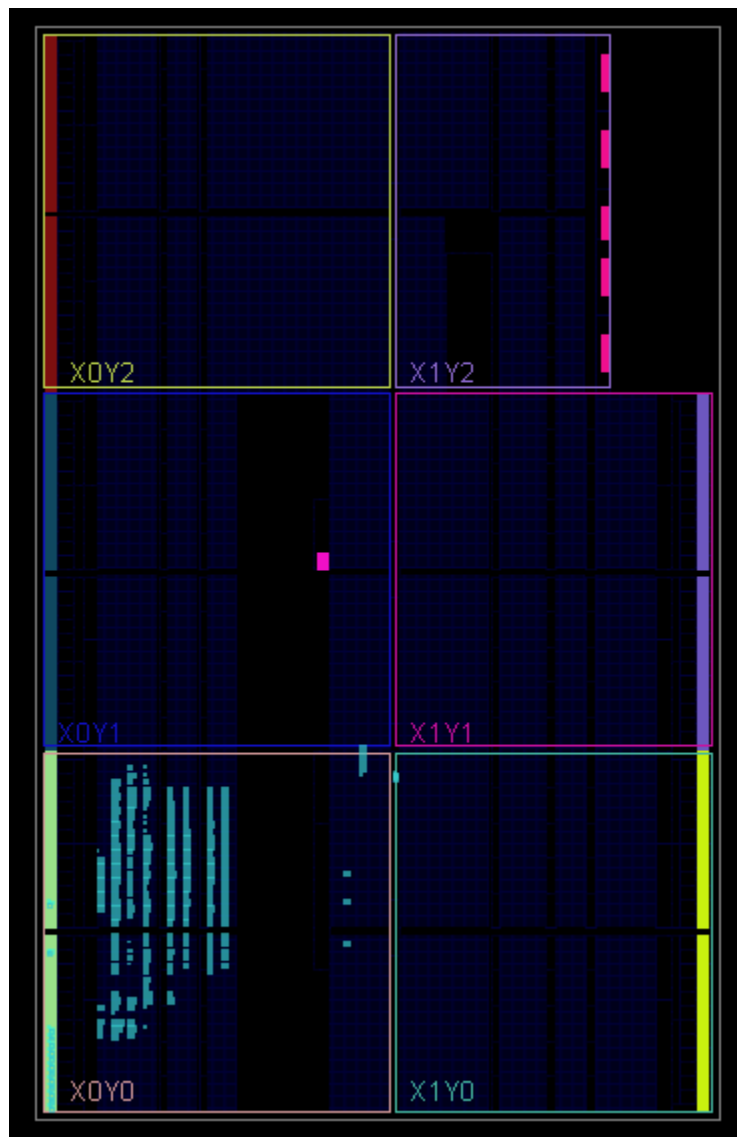


Figure 9.2 — Post-implementation device (floorplan) view.

10. Repository Structure

The repository is organized by functional block, mirroring the design hierarchy described in Section 2:

```
SoC-Design-Connecting-RISC-V-Processor-with-Multiple-peripherals-using-APB-Bus/
├── README.md
├── SoC TOP/
│   ├── SoC.v                      (top-level integration)
│   ├── RISC_APB_Wrapper.v        (bridge FSM)
│   ├── APB_Master.v
│   ├── APB_Decoder.v
│   ├── SoC_tb.sv                (top-level testbench)
│   ├── SoC_DO.do                (ModelSim/Questa run script)
│   └── Processor/               (RISC-V single-cycle core)
│       ├── RISC_V_Wrapper.v, Control_Unit.v, ALU.v, Reg_File_Model.v,
│       ├── Extention_Unit.v, Data_Mem.v, Inst_Memory.v, PC_clk.v,
│       ├── Adder4.v, Adder_PC.v, Mux2_1.v, Mux3_1.v, RISC_V_tb.v
│       └── Inst_Mem.dat, D_Mem.dat, Registers.dat (memory init files)
├── UART Peripheral/
│   ├── UART_TOP.v, Baud_Rate_Generator.v, UART_tb.sv
│   ├── Tx Unit/ (Tx_Unit.v, Tx_FIFO.v, Tx_Transmitter.v, Parity_Selector.v)
│   └── Rx Unit/ (Rx_Unit.v, Rx_FIFO.v, Rx_Receiver.v, Error_Detector.v)
└── Structure and Testing/        (architecture + waveform + Vivado screenshots)
```

Design files are commented in-line at the point of use; two dedicated .do scripts (for the Tx FIFO and Rx FIFO individually) are also included in the repository to speed up focused re-simulation of the UART sub-blocks during development.

11. Future Work

Per the project README, the following extensions are planned for subsequent releases:

- Additional peripherals connected on the already-reserved address ranges (2000–3000 and 3000–4000), exercising the APB decoder's remaining decode paths.
- Pipelining the RISC-V core to improve throughput beyond the current single-cycle implementation.